

Object-Oriented Programming (OOPs)

Comprehensive Theory, Core Concepts, Code Examples & 25 Practice Questions

1. Introduction & OOPs Theory

Object-Oriented Programming (OOPs) is a programming paradigm based on the concept of **objects**, which can contain data (attributes/properties) and code (methods/functions). OOPs aims to implement real-world entities like inheritance, hiding, polymorphism, etc. in programming.

Key OOPs vs Procedural Programming

- **Procedural Programming (POP):** Focuses on functions and procedure execution (e.g., C, Pascal). Data moves freely around the system.
- **Object-Oriented Programming (OOP):** Focuses on data security and objects (e.g., C++, Java, Python). Programs are organized around objects rather than logic.

2. Core Concepts & Code Examples

1. Class & Object

Class: A user-defined data type or blueprint that defines properties and behaviors.

Object: An instance of a class that holds actual values in memory.

```
class Car {
public:
    string brand;
    void drive() { cout << "Driving " << brand; }
};
Car myCar; myCar.brand = "Toyota"; myCar.drive();
```

2. Encapsulation

Wrapping data (variables) and code (methods) together into a single unit (class) and restricting direct access using private access specifiers.

```
class BankAccount {
private:
    double balance;
public:
    void setBalance(double b) { balance = b; } // Setter
    double getBalance() { return balance; } // Getter
};
```

3. Abstraction

Hiding complex internal implementation details and displaying only essential features to the user (achieved via Abstract Classes or Interfaces).

```
class ATM {  
public:  
    virtual void withdrawMoney() = 0; // Abstract function  
};
```

4. Inheritance

The mechanism where a new class (derived/child class) inherits properties and methods from an existing class (base/parent class).

- **Single:** $A \rightarrow B$
- **Multilevel:** $A \rightarrow B \rightarrow C$
- **Multiple:** $A + B \rightarrow C$
- **Hierarchical:** $A \rightarrow B, A \rightarrow C$
- **Hybrid:** Combination of two or more types.

```
class Animal { public: void eat() { cout << "Eating"; } };  
class Dog : public Animal { public: void bark() { cout << "Barking"; } };
```

5. Polymorphism

Ability of a message or function to be processed in more than one form ("Poly" = many, "Morph" = forms).

- **Compile-Time (Static):** Function Overloading & Operator Overloading.
- **Run-Time (Dynamic):** Function Overriding (using virtual functions).

```
// Compile-time Overloading  
int add(int a, int b) { return a + b; }  
double add(double a, double b) { return a + b; }
```

3. Additional Important Concepts

- **Constructor:** A special member function called automatically when an object is instantiated. Has the same name as the class.
- **Destructor:** A function called automatically when an object is destroyed to release memory.
- **Association, Aggregation, & Composition:** Relationships defining how objects interact (e.g., "Has-A" relationship).

4. OOPs Practice Questions (25 MCQs)

1. Which of the following is NOT one of the 4 main pillars of OOPs?

- A) Encapsulation
- B) Compilation
- C) Polymorphism
- D) Inheritance

2. Which concept wraps data and methods into a single unit?

- A) Abstraction
- B) Inheritance
- C) Encapsulation
- D) Polymorphism

3. What is a Blueprint or Template from which objects are created?

- A) Function
- B) Class
- C) Structure
- D) Interface

4. Function Overloading is an example of which type of Polymorphism?

- A) Run-time Polymorphism
- B) Compile-time Polymorphism
- C) Dynamic Binding
- D) Late Binding

5. Which feature of OOPs allows showing only necessary details to the user?

- A) Abstraction
- B) Data Hiding
- C) Encapsulation
- D) Polymorphism

6. What is automatically called when an object is created?

- A) Destructor
- B) Main method
- C) Constructor
- D) Static block

7. Which type of inheritance involves a child class deriving from multiple parent classes?

- A) Multilevel Inheritance
- B) Multiple Inheritance
- C) Hierarchical Inheritance
- D) Single Inheritance

8. Which access specifier makes members accessible only within the same class?

- A) public
- B) protected
- C) private
- D) default

9. Function Overriding occurs in which scenario?

- A) Within the same class
- B) Between Parent and Child classes
- C) Outside any class
- D) In global functions

10. Can a Constructor have a return type in Java/C++?

- A) Yes, void
- B) Yes, int
- C) No, constructors have no return type
- D) Yes, object reference

11. What is the process of freeing memory when an object goes out of scope called?

- A) Instantiation
- B) Destruction
- C) De-allocation
- D) Garbage Collection / Destructor call

12. Which keyword is used to refer to the current instance of a class in Java/C++?

- A) super
- B) this
- C) self
- D) base

13. Which concept is represented by a "HAS-A" relationship?

- A) Inheritance
- B) Aggregation / Composition
- C) Polymorphism
- D) Abstraction

14. What is an Abstract Class?

- A) Class that cannot be instantiated
- B) Class with only private members
- C) Class without any methods
- D) Class that cannot be inherited

15. What is the "IS-A" relationship in OOPs?

- A) Association
- B) Encapsulation
- C) Inheritance
- D) Composition

16. Which operator cannot be overloaded in C++?

- A) +
- B) :: (Scope Resolution)
- C) ==
- D) =

17. What is a Virtual Function used for in C++?

- A) Static Binding
- B) Dynamic / Run-Time Polymorphism
- C) Data Security
- D) Private Access

18. What is a Copy Constructor?

- A) Constructor creating new blank object
- B) Constructor initializing object using another object of same class
- C) Constructor destroying objects
- D) Static constructor

19. Which access specifier allows members to be accessed in derived classes but not outside?

- A) private
- B) public
- C) protected
- D) internal

20. What is a Pure Virtual Function in C++?

- A) Function with no return type
- B) Virtual function assigned to 0 (= 0)
- C) Function inside private block
- D) Function with default parameters

21. Strong form of Aggregation where child object's lifecycle depends on parent is called?

- A) Composition
- B) Inheritance
- C) Association
- D) Encapsulation

22. Can an interface contain instance variables in standard Java?

- A) Yes
- B) No, only public static final constants
- C) Yes, if private
- D) Yes, if protected

23. Which symbol is used for Destructor in C++?

- A) #
- B) ~ (Tilde)
- C) *
- D) &

24. What is Garbage Collection in OOPs languages like Java?

- A) Deleting unreferenced objects automatically
- B) Cleaning source code
- C) Removing duplicate variables
- D) Optimizing loops

25. Method Overloading requires functions to have:

- A) Same name, same parameters
- B) Same name, different parameter signature
- C) Different name, same parameters
- D) Same return type only

Answer Key

- | | | | | |
|------|-------|-------|-------|-------|
| 1. B | 6. C | 11. D | 16. B | 21. A |
| 2. C | 7. B | 12. B | 17. B | 22. B |
| 3. B | 8. C | 13. B | 18. B | 23. B |
| 4. B | 9. B | 14. A | 19. C | 24. A |
| 5. A | 10. C | 15. C | 20. B | 25. B |